Abstract: Secure and Globally Distributed Static Website on AWS Using Terraform

Mahmoud Kanaan | Matriculation Number 92133250 | DLBSEPCP01_E | Task 3

Background and objective

Static websites avoid a continuously running application server, but they still need dependable delivery, secure transport, and reproducible deployment. This project addressed a focused problem: publish one self-contained HTML page worldwide without exposing its storage origin or adding unnecessary compute services. The objective was to store index.html in a private Amazon S3 bucket and distribute it through Amazon CloudFront over HTTPS. The design also had to provide high availability, low latency, managed scaling, and secure removal after assessment. Terraform describes the complete environment so the same reviewed configuration can create, update, verify, and destroy it consistently.

Architecture concept

A global user opens the CloudFront distribution domain. CloudFront redirects HTTP to HTTPS and checks its edge cache. It returns a cached copy when possible; otherwise, it sends a signed request to the regional S3 origin through Origin Access Control (OAC). OAC uses AWS Signature Version 4 and lets the bucket policy recognize the specific distribution. The private bucket is not an S3 website endpoint and contains only index.html. Its policy grants the CloudFront service principal only s3:GetObject when AWS:SourceArn matches the created distribution. Terraform provisions and links the bucket, object, OAC, distribution, and policy.

Technical implementation

The code requires Terraform 1.6.0 or later and constrains the AWS provider to 6.x. Variables define eu-central-1 and a validated lowercase project name. The AWS account ID makes the bucket name globally unique. Separate resources configure BucketOwnerEnforced ownership, all Public Access Block controls, versioning, and AES256 encryption. The object reads site/index.html, declares UTF-8 HTML, sets five-minute caching, and uses filemd5() to detect changes. CloudFront enables IPv6, compression, redirect-to-https, the managed CachingOptimized policy, PriceClass_All, and its default TLS certificate.

Security approach

Security uses several reinforcing controls. BucketOwnerEnforced disables ACLs, and all four Public Access Block values are true. The bucket policy grants no anonymous access: it permits only the CloudFront service principal, s3:GetObject, the object path, and the distribution-specific source ARN. Viewers use HTTPS, while OAC signs origin requests. SSE-S3 protects the object at rest and versioning aids recovery. Credentials come from the local AWS authentication context rather than source files. Terraform state, plans, .env files, private keys, assignment material, and the knowledge base are excluded from version control and final packaging.

Availability and scaling

Availability and scale come from managed services rather than a server fleet. S3 provides a redundant regional object origin without an operating system, runtime, or storage node to maintain. CloudFront adds a global edge layer that serves cached content near users and reduces repeat requests to S3. No instance sizing, autoscaling group, or load balancer is required. PriceClass_All prioritizes worldwide reach over restricting delivery to lower-cost regions. The architecture therefore has no fixed web-server capacity: AWS manages storage and distribution while caching, compression, and edge delivery improve efficiency for the static object.

Deployment and verification

Deployment followed Terraform initialization, formatting, validation, planning, and application. The lock file was retained, the reviewed plan contained only expected S3 and CloudFront resources, and the initial apply added nine resources without changes or destruction. The live URL returned HTTP 200 with CloudFront headers. HTTP redirected to HTTPS, all Public Access Block values were true, and direct unauthenticated S3 access returned HTTP 403. CloudFront reported Deployed, repeated requests produced Hit from cloudfront evidence, and a final Terraform plan reported no differences between the configuration and deployed infrastructure.

Challenges and corrections

Practical issues were corrected during the work. Git initially detected embedded repositories, so nested metadata was removed while the intended parent repository was retained. Assignment and knowledge-base files were explicitly excluded. A plan export failed from the parent directory because the evidence path was absent; running it inside iu-cloud-programming resolved the path. Screenshots were standardized in one folder, and tests were marked PASS only after genuine output existed. For repeatability testing, a visible status message changed index.html, Terraform proposed only the S3 object update, and a completed CloudFront invalidation exposed the new content.

Result and conclusion

The webpage is available through CloudFront HTTPS while S3 remains private. CloudFront is the only permitted reader of index.html; direct S3 access is denied, HTTP redirects to HTTPS, and cache-hit evidence confirms edge caching. Terraform validation passes, content updates are repeatable, and the final plan is idempotent. The smallest suitable service set meets the requirements: S3 stores the object, CloudFront distributes it, OAC protects origin access, and Terraform controls the lifecycle. EC2, load balancers, containers, APIs, databases, custom DNS, and backend code would add cost and maintenance without serving this one-page site. The result is secure, global, reproducible, and removable after assessment.